INSTALLATION

HORIZON GRID

Windows Installation Guide

Version: 0.2.5

Documentation prepared: 2026-08-23

PREPARED FOR EVALUATION

Table of Contents

HORIZON GRID Standalone Windows Installation — A Complete Walkthrough

Why this document exists

`user-03-installation.md` and `tech-06-windows-deployment.md` already cover the installer from the administrator's point of view and from the architecture's point of view, respectively. This document sits between them: a single, start-to-finish walkthrough of a real standalone Windows install — every Setup Wizard page in the order it actually appears, every Start Menu shortcut, where data actually lives on disk, how to upgrade or reconfigure later, how to uninstall (including the destructive path), and a real installer bug that was found and fixed during this project. It draws directly from `windows/installer.iss` and `windows/wizard/Setup-Wizard.ps1` — nothing here is inferred from the installer's general shape.

System Requirements

The installer's own `[Code]`-section prerequisite check (`Check-Prerequisites.ps1`) verifies these before copying any files, and the same script backs the "Diagnostics" shortcut's `prerequisites.txt` output for after-the-fact troubleshooting:

Requirement	Detail	Hard or soft check
OS	64-bit Windows 10 (build ≥ 10240) or Windows 11	Hard
Privileges	Running as Administrator	Hard
RAM	At least 8 GB total system RAM	Soft — flagged but not blocking
Disk space	At least 8 GB free on the system drive	Hard
Docker Desktop	Installed (`docker.exe` on PATH)	Hard
Docker Desktop	Actually running (`docker info` succeeds — checked this way, not by process/service name, because WSL2-backed Docker Desktop doesn't always run the legacy `com.docker.service` Windows service)	Hard
Docker Compose	v2 plugin available (`docker compose version`)	Hard
Ports	`3000, 8000, 5433, 6379, 7475, 7688, 9200` free	Soft — the wizard's own Port Review page lets you remap any conflict

A failed hard check does not silently abort the install. It surfaces a "Continue anyway?" message box, so an administrator who understands the risk can still proceed — the check is advisory-with-friction, not an unbypassable gate. The installer also restricts itself to 64-bit-compatible systems at the package level (`ArchitecturesAllowed=x64compatible` / `ArchitecturesInstallIn64BitMode=x64compatible` in `installer.iss`), so the same 64-bit requirement is effectively checked twice — once by the installer package itself, once again by this script.

Docker Desktop is the one prerequisite you have to solve yourself before running the installer: HORIZON GRID's real services (Postgres, Redis, Neo4j, OpenSearch, the backend, the frontend, and the Celery worker/beat processes) run as Docker containers, and nothing

about the installer replaces that with a native Windows service. See `tech-06-windows-deployment.md` for the full architectural rationale.

The Installer File

The compiled installer follows a fixed naming pattern set by `installer.iss`'s `OutputBaseFilename`:

```
HORIZON-GRID-Setup-<version>.exe
```

For example, the current build produces `HORIZON-GRID-Setup-0.2.2.exe`. The visible product name throughout the installer's own UI (title bar, publisher field, Start Menu group) is "HORIZON GRID" — but the installer deliberately does **not** derive the on-disk installation folder name from that branding. It installs to:

```
%ProgramFiles%\IOC Intelligence Platform\
```

This is intentional, not an oversight: every PowerShell script the wizard and Start Menu shortcuts run afterward (`Common.ps1`'s `$script:InstallDir` / `$script:DataDir`) resolves this exact folder name independent of anything in `installer.iss`. If the visible product name and the on-disk folder name were the same variable, a fresh install would land under `Program Files\HORIZON GRID` while every script installed alongside it kept looking for `Program Files\IOC Intelligence Platform` — breaking the install it had just performed, not just cosmetically mismatching a name. Section "Data Location" below covers the same reasoning for `ProgramData`.

Running the Installer

`user-03-installation.md` already covers the first-stage Inno Setup screens (destination folder, optional desktop icon task, ready-to-install summary, file copy) in full — that flow is unchanged here and is not repeated. Two details worth calling out precisely because they're easy to miss:

- The prerequisite check described above runs as part of clicking through the Inno Setup pages, before the real file copy — so the "Continue anyway?" prompt, if it appears, comes before anything is written to disk.
- Closing the installer's final "Completing Setup" screen with the "Launch the setup wizard now" box checked (checked by default) immediately launches `Setup-Wizard.ps1` — this is a separate, second-stage application, not another Inno Setup page. Everything from here on in this document is that second stage.

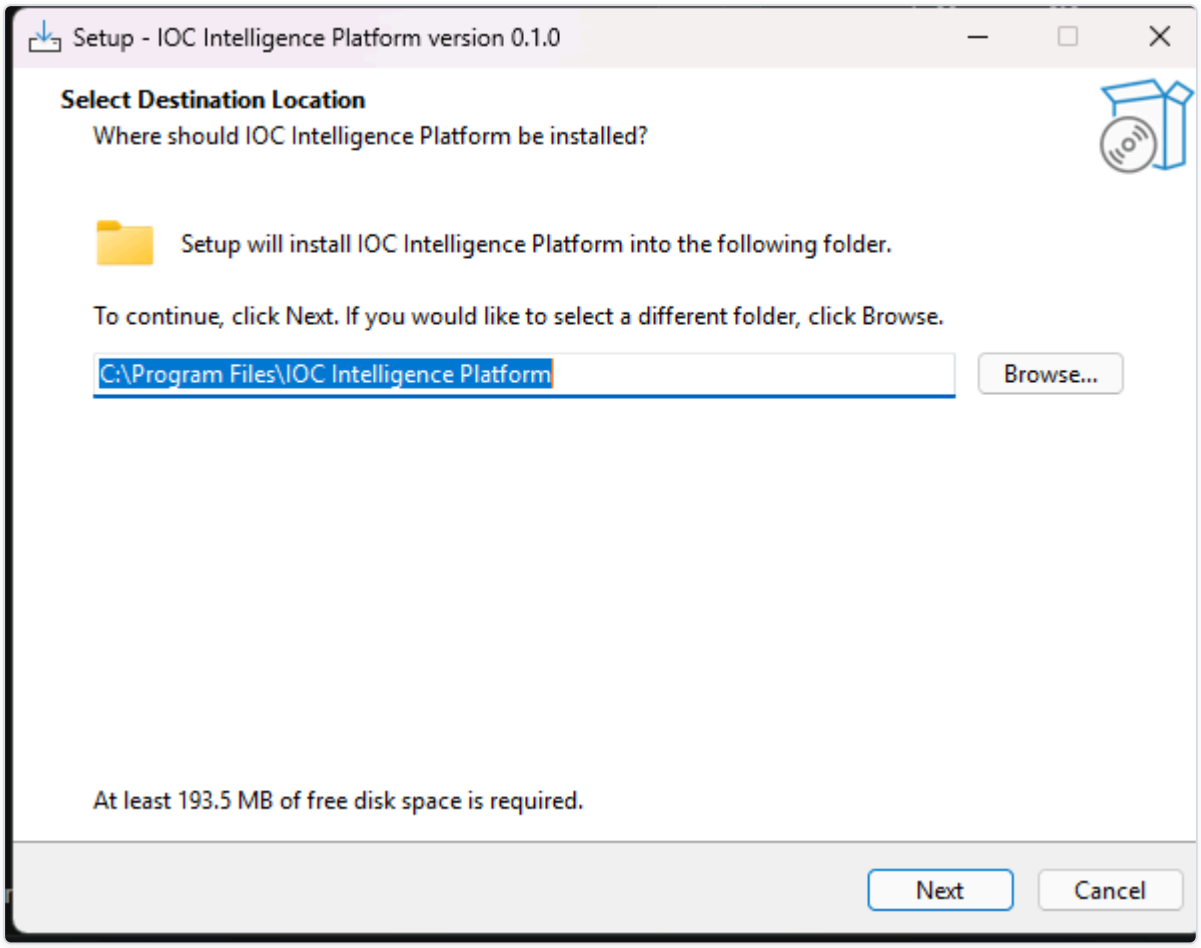

Figure 1 — The Inno Setup installer's file-copy progress, immediately before it hands off to the Setup Wizard.

The Setup Wizard, Page by Page

The Setup Wizard is a real WinForms desktop application (`windows/wizard/Setup-Wizard.ps1`) — not a web page, not part of the Inno Setup UI. On launch it checks whether it's running with a genuinely elevated token and, if not, re-launches itself via a UAC prompt: being a member of the Administrators group is not sufficient on its own, because Windows gives a normally-launched process (including one started from a Start Menu shortcut, or from the installer's own postinstall step) a filtered token that cannot pass the ACL grants the wizard needs to make later. Declining that UAC prompt ends the wizard with an explanatory message box rather than continuing in a broken, can't-write-anything state.

The page sequence is fixed:

Welcome → Administrator Account → AI Configuration → Threat Intelligence Providers → Network Ports → Ready to Install → Setup Complete

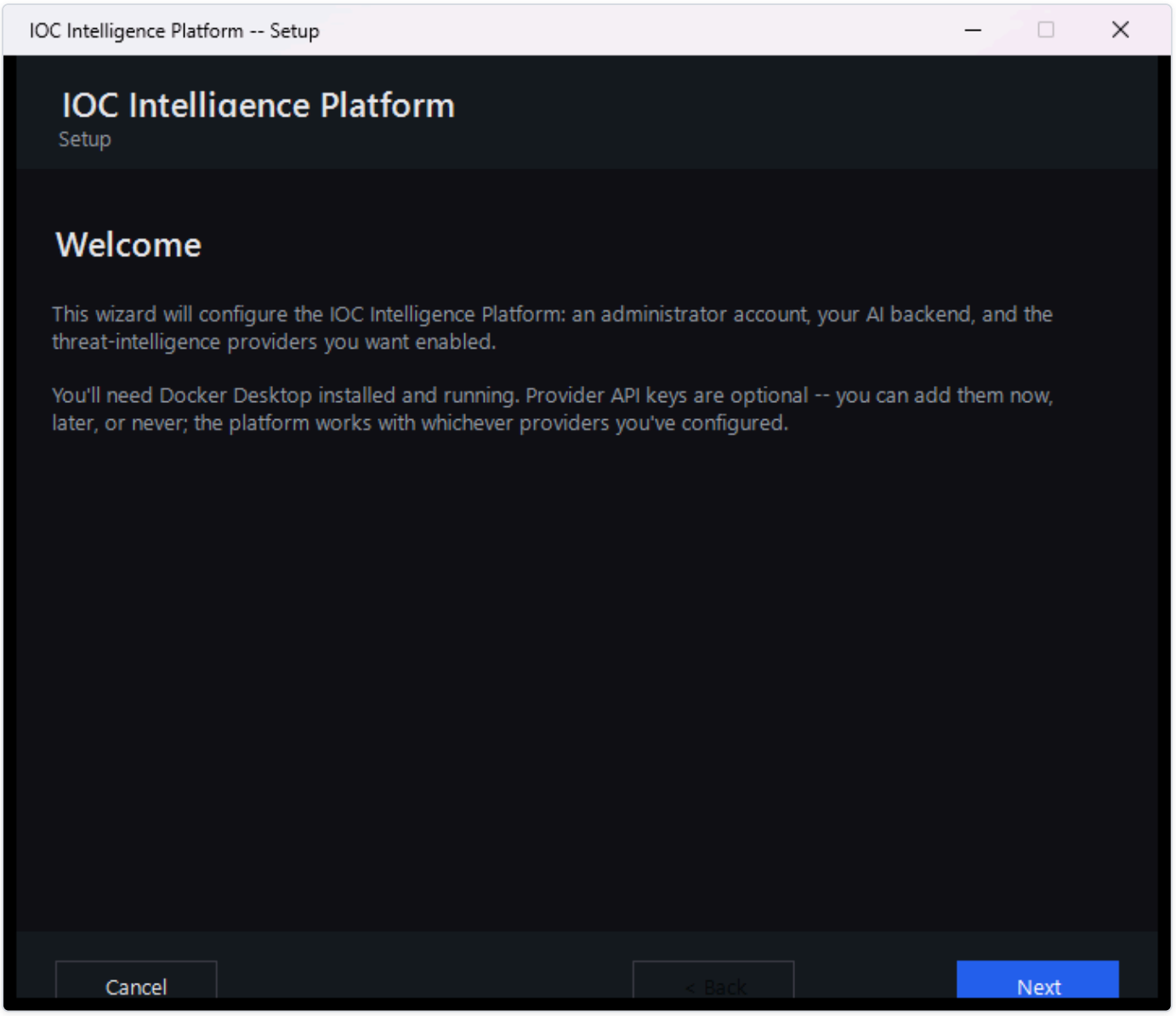

Figure 2 — The Setup Wizard's Welcome page on a fresh install.

Welcome

States what the wizard is about to collect (administrator account, AI backend, provider keys) and reminds you that Docker Desktop needs to be installed and running, and that every provider key is optional — the platform works with any subset configured, including none. On a machine that already has the platform installed, this page's title and text change to "Reconfigure HORIZON GRID" and explicitly reassure you that your investigation data, cases, and history are untouched by anything in this wizard.

Administrator Account

Collects the email, optional full name, and password (8-64 characters, with confirmation) for the account the wizard will register once the stack is running. There is no role picker on this page, because there doesn't need to be one: the backend's own registration rule automatically grants the Admin role to the very first account ever registered against a fresh database, and rejects every registration attempt after that with `403 Forbidden`. The email field is validated against a deliberately strict pattern matched to real cases the

backend's own email validator rejects (a trailing period before the `@`, consecutive periods, over-length local parts, and unquoted special characters) — catching those here means you see "Enter a valid email address" on this page instead of a raw "Account creation failed" error much later, on the Ready to Install page.

On a reconfigure run, an existing admin account is assumed to already exist, and this page explains that leaving both fields blank keeps it unchanged. Re-entering your existing email and password here does not create or change anything — it only signs you in for this wizard session, which is what makes the live "Test" buttons on the following two pages work. Skip this and every Test button on those pages will show a "sign-in required" message rather than actually testing anything.

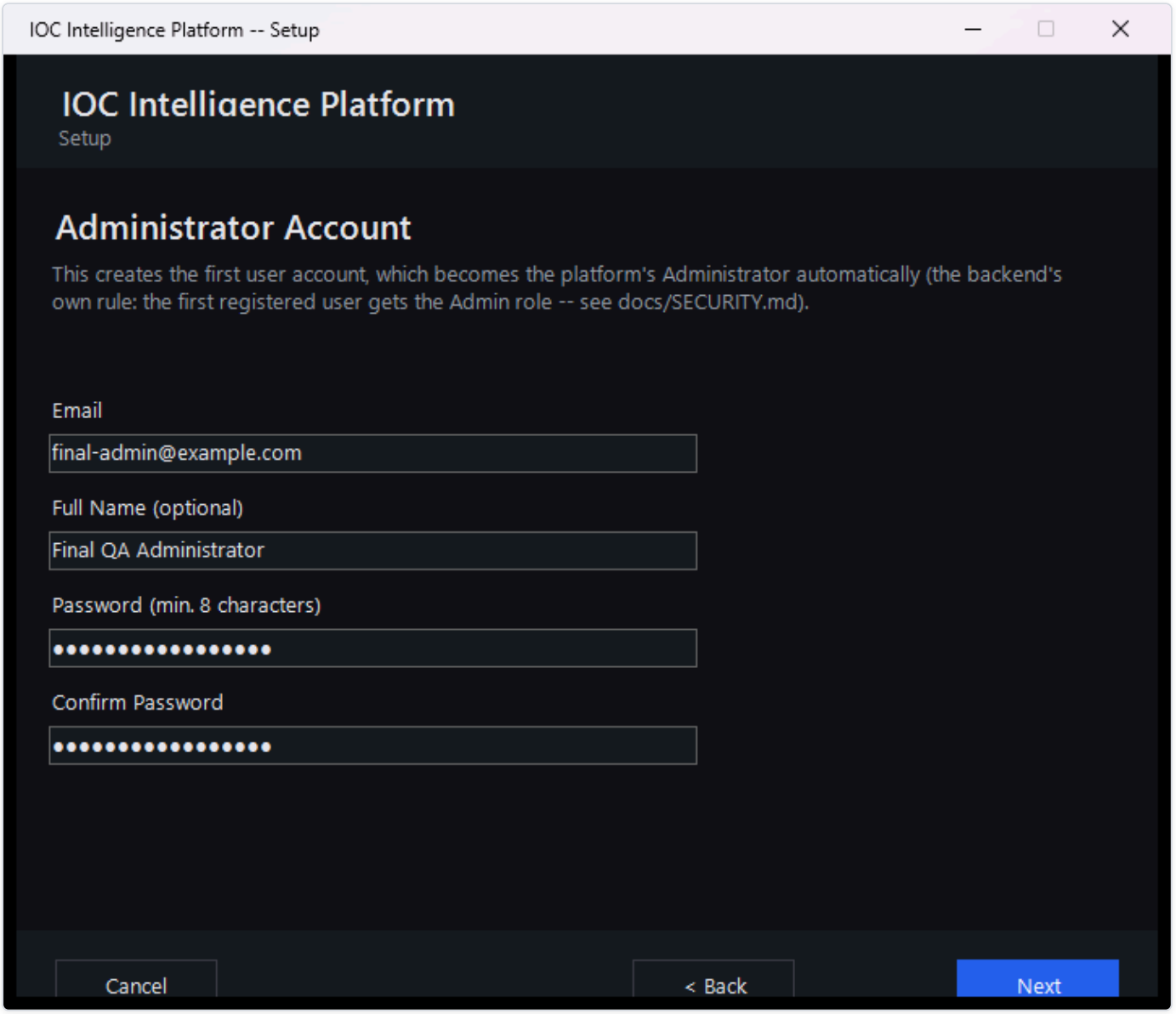

Figure 3 — The Administrator Account page, showing the email/name/password fields.

AI Configuration

Lets you pick which of the platform's eleven supported AI backends to use — Ollama (local, no API key, no cost), Anthropic, AWS Bedrock, Google Gemini, Groq, OpenAI, Kimi, DeepSeek, xAI, Mistral, or OpenRouter — and enter the corresponding credentials. Each backend's panel has its own live "Test Connection" button that calls the real running backend's `POST /api/v1/ai/test` endpoint (once

a session exists, per the note above) and reports back the actual model used and the round-trip latency, or the actual failure message if it didn't work. The Groq panel's model field is a live-refreshing combo box rather than a fixed dropdown: leaving the API key field and returning to it (a focus-out event, not every keystroke) queries Groq's own `/models` endpoint through the backend and repopulates the list, so it doesn't go stale as Groq adds or retires models. This page can be changed at any time later by re-running Configuration — switching AI backends does not require reinstalling anything, matching the runtime-switchable design described elsewhere in this documentation set.

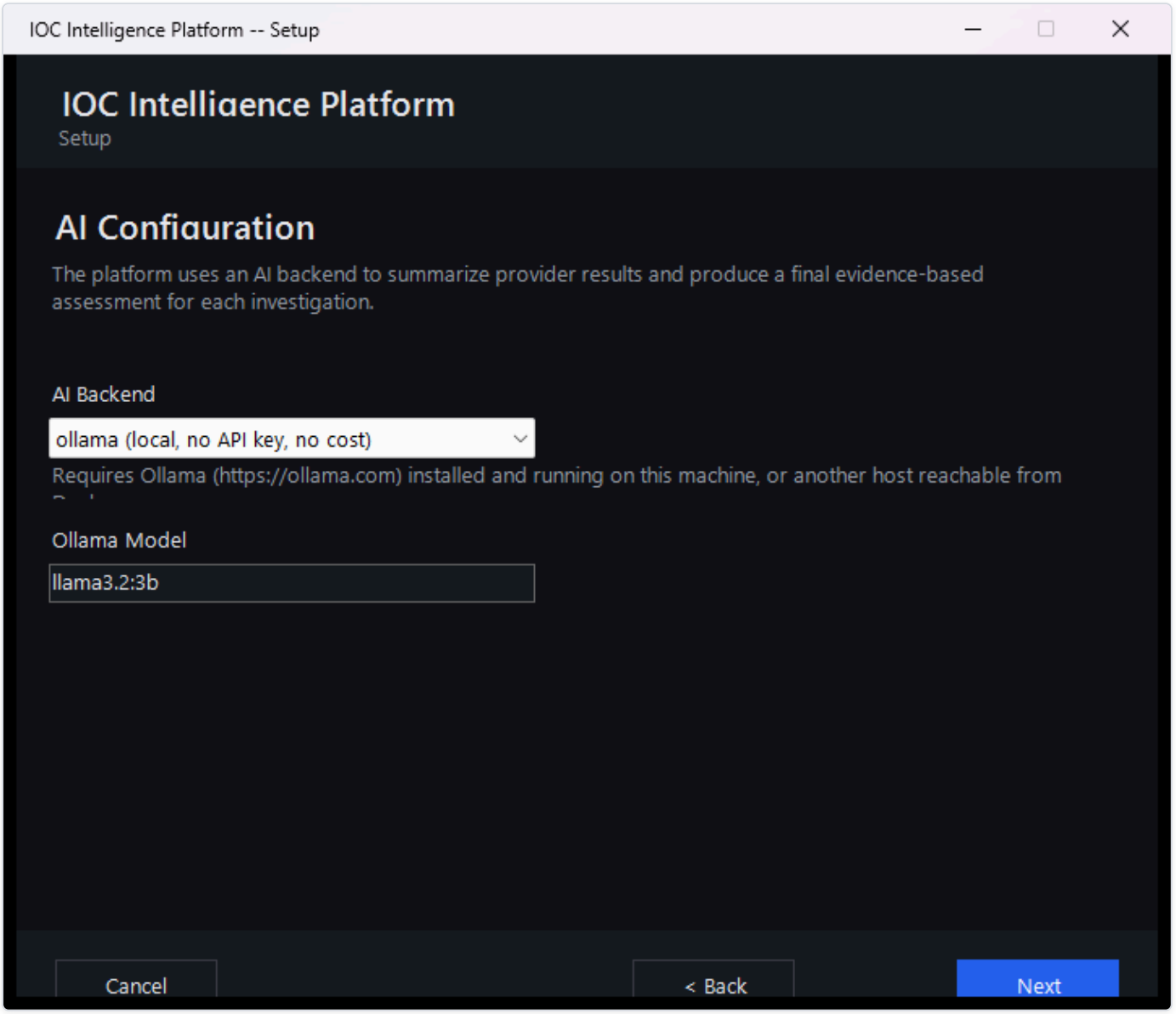

Figure 4 — The AI Configuration page with the Anthropic panel selected.

Threat Intelligence Providers

Presents 8 provider cards, each with a link to where to obtain a key and a live "Test" button: VirusTotal, AbuseIPDB, AlienVault OTX, the combined abuse.ch group (one shared Auth-Key covers URLhaus, ThreatFox, and MalwareBazaar — 3 providers under one card), NIST NVD, Hybrid Analysis, Censys (which needs both a Personal Access Token and an Organization ID), and PhishTank. Between them, these 8 cards cover 10 of the platform's 18 registered providers. Every field on this page is optional, stated explicitly at the top of the page — leaving a key blank simply skips that provider, and the platform works correctly with any subset configured, including none at all.

The remaining 8 registered providers do not appear on this page at all:

- crt.sh, CISA KEV, MITRE ATT&CK, WHOIS/RDAP, Spamhaus, and the internal OSINT collector require no credential at all to use, so there is nothing on this page for them to configure.
- urlscan.io and Google Safe Browsing do require an API key but are not represented on this wizard page — they can be configured after setup from the platform's own Providers page in the web interface, which drives credentials for every registered IOC provider generically rather than duplicating this wizard's per-provider layout.

PhishTank itself appears on this page even though no key is strictly required for it, since an optional key raises its rate limit.

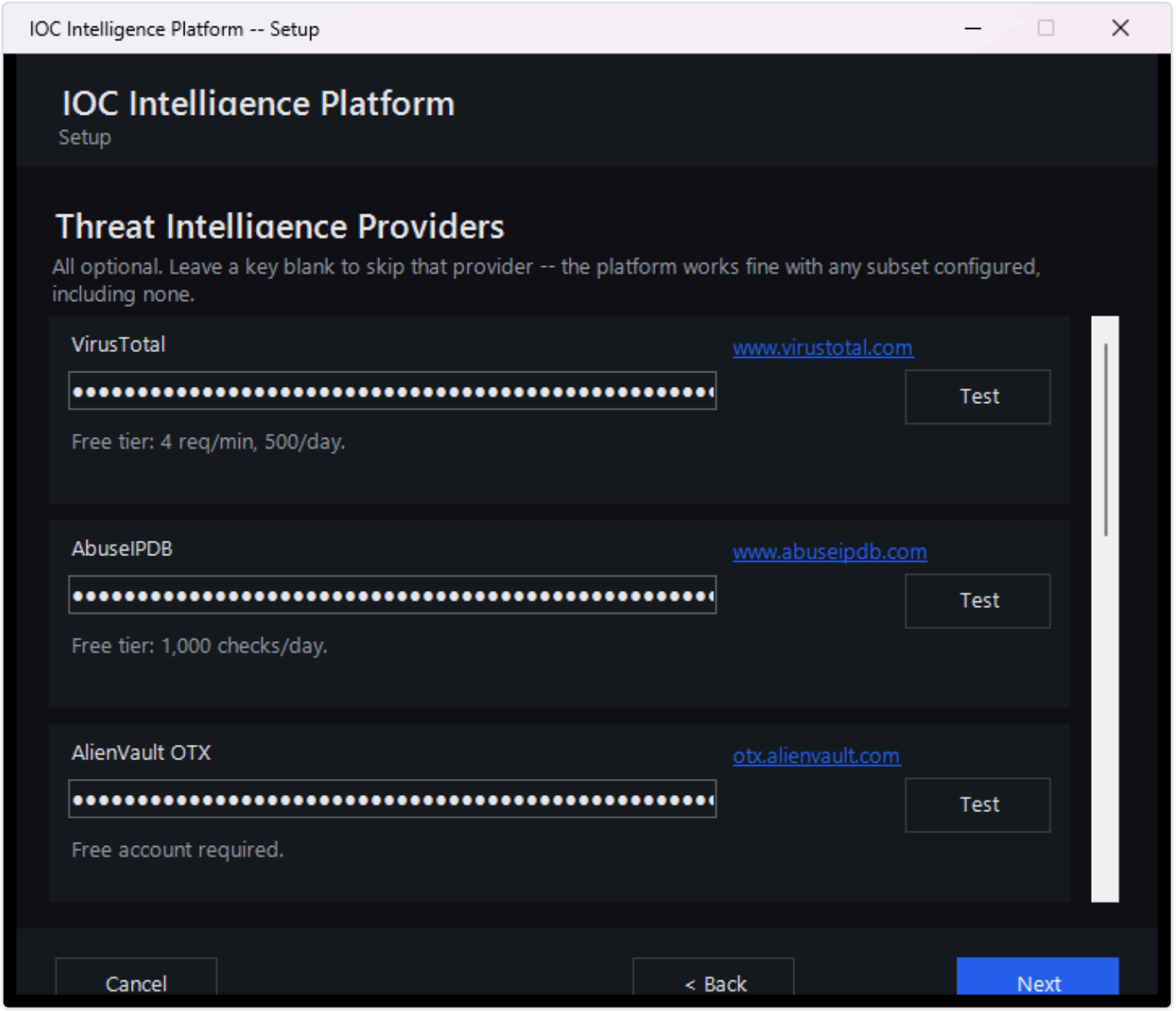

Figure 5 — The Threat Intelligence Providers page, scrolled to show several provider cards with Test buttons.

Network Ports

Lists every host port the stack needs — the web interface, the backend API, and the internal Postgres/Redis/Neo4j/OpenSearch ports — pre-filled with the platform's defaults (3000, 8000, 5433, 6379, 7475, 7688, 9200) and flags any that are already in use on this machine,

suggesting the next free port as a starting point. Validation requires every port to be a distinct value between 1024 and 65535; a duplicate or out-of-range entry is rejected with a specific error before you can move on.

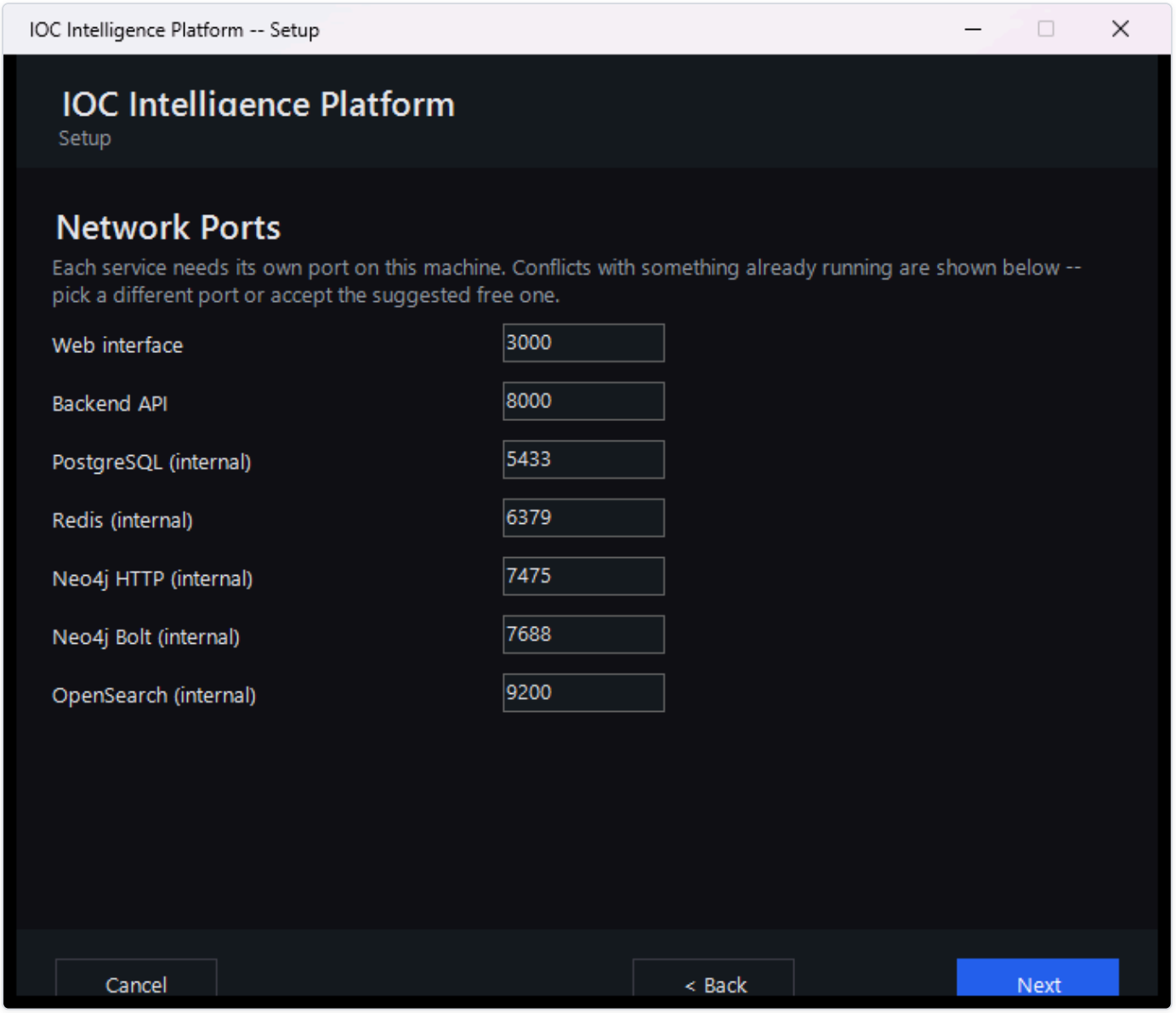

Figure 6 — The Network Ports page showing a port-conflict warning next to one field.

Ready to Install

Summarizes what's about to happen (which admin account, which AI backend, how many providers configured out of the total, and the URLs the platform will be reachable at — including a best-effort LAN address for other devices on the network) and provides the "Start Installation" button plus a live scrolling progress log. Clicking it runs the following, in order, with every step's real outcome (success or specific failure) written to that log rather than left to guesswork:

1. On a reconfigure run only, backs up the database first (see "Upgrade / Reconfigure" below).
2. Detects this machine's LAN-facing IPv4 address for display purposes (best-effort; the platform still works locally if this fails).
3. Writes the real `.env` configuration file and applies the fresh-install stale-volume check described below.

4. Creates a Windows Firewall rule scoped to Private networks for the frontend and backend ports.
5. Runs `docker compose up -d --build`, which can take several minutes on first run while images build.
6. Polls the backend's `/health` endpoint for up to 3 minutes.
7. Registers the administrator account (or signs in, if it turns out one already exists with that email) against the now-running backend.

Any failure at any step — including one this list doesn't anticipate — is caught and written to the log with a real, specific message rather than leaving the wizard silently stuck; the Back and Start Installation buttons re-enable so you can fix the underlying issue and retry without restarting the entire wizard.

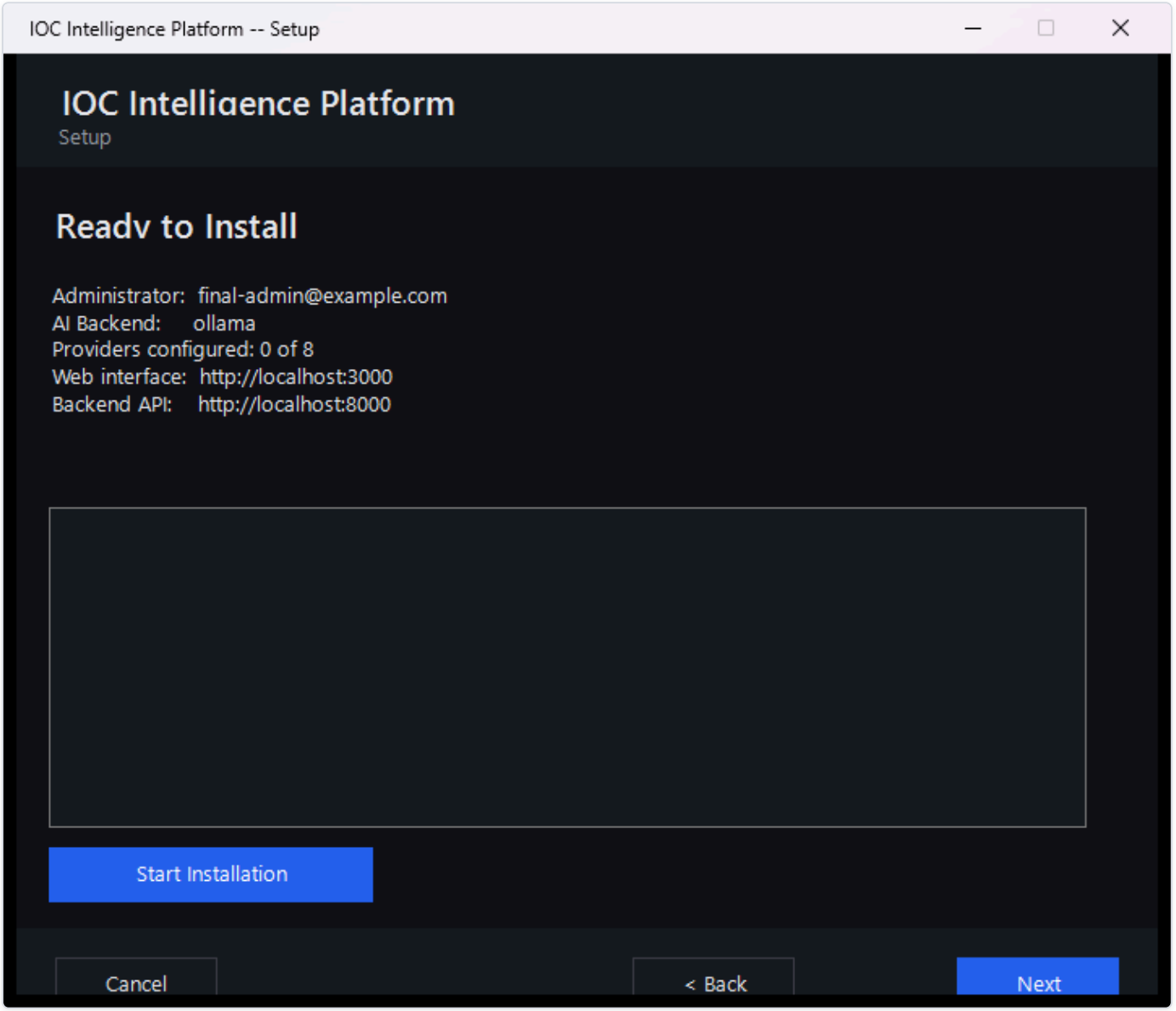

Figure 7 — The Ready to Install page mid-installation, showing the scrolling progress log.

Setup Complete

Shows the local URL (`http://localhost:<port>`) and, if one was detected, the LAN URL other devices on the same Private network can use, plus a checkbox (checked by default) to open the platform in your browser immediately when the wizard closes.

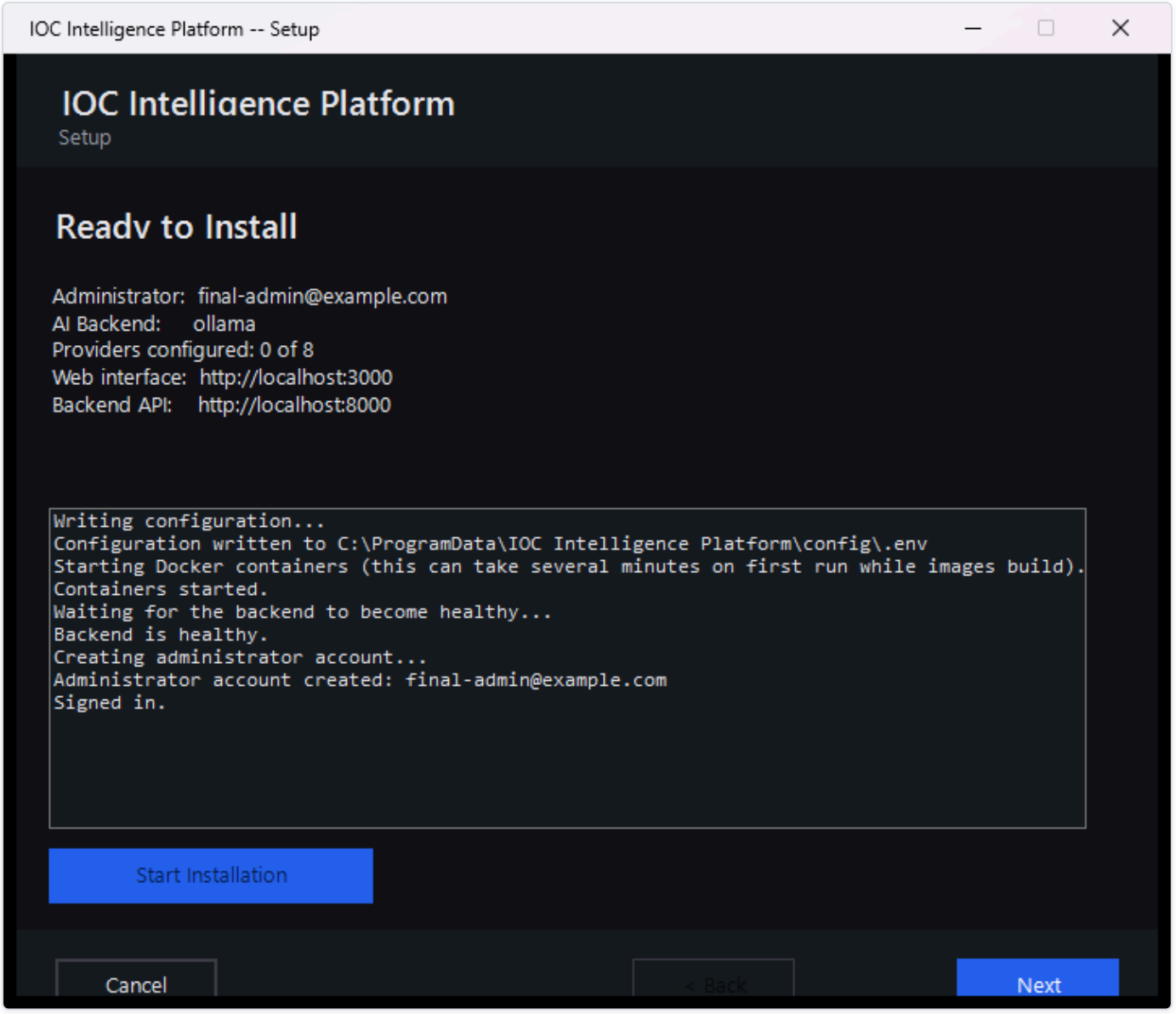

Figure 8 — The Setup Complete page showing both the local and LAN access links.

First Launch

If you left "Open the platform now" checked, your browser opens straight to the sign-in page once the wizard window closes. If not, or if you're returning later, use the **Open Platform** shortcut described below — it's the reliable way to start the platform from a cold boot, not just a bookmark.

Start Menu Shortcuts

`installer.iss`'s `[Icons]` section creates one Start Menu group containing:

Shortcut	What it actually does
Open Platform	Runs `Open-Platform.ps1`. If the platform is already up and healthy, it skips straight to opening the browser with no UAC prompt at all. Otherwise it shows a small progress window, starts Docker Desktop itself if it isn't already running (waiting up to 3 minutes, since this is the normal case right after a reboot), starts the containers, waits for the backend to report healthy, and only then opens the browser — so this one shortcut is the entire recovery story after a restart, with no separate manual steps.
Configuration	Re-runs `Setup-Wizard.ps1`. On an existing install this is the reconfigure path described below.
Start Platform	Runs `Service-Start.ps1`: `docker compose up -d`, then polls `/health`.
Stop Platform	Runs `Service-Stop.ps1`: `docker compose stop` — no `-v` flag, so container volumes (and therefore all data) are left untouched.
Restart Platform	Runs `Service-Restart.ps1`: `docker compose restart`, then polls the backend's health for up to 2 minutes and reports whether it came back up in time.
Service Status	Runs `Service-Status.ps1`: prints real `docker compose ps` container status, live backend/frontend health-check results, and whether Docker Desktop itself is currently running — a single console screen for "what's actually going on right now."
Backup Database Now	Runs `Backup-Database.ps1` on demand — the same `pg_dump`-inside-the-running-Postgres-container mechanism that runs automatically before a reconfigure (see below), producing a timestamped `.sql` file in the backups folder. If the Postgres container isn't running, it says so and exits cleanly rather than failing.
Diagnostics	Runs `Diagnostics.ps1`, producing a zipped diagnostic bundle on the Desktop containing container status, recent per-service logs, the setup log, prerequisite-check output, and a redacted copy of the configuration (every value replaced with `***REDACTED***(set, N chars)` or `(not set)` — variable names are kept so you can confirm a key is present without ever exposing it). The bundle also runs a second, shape-based redaction pass over collected logs to catch secrets that might appear with no label nearby at all (bearer tokens, `sk-ant-...` and other API-key shapes, AWS access keys, JWTs, and credentials embedded in connection strings) — worth reviewing yourself before sharing the bundle with anyone, exactly as the tool's own closing message says.
Documentation	Opens the documentation copied in at install time.
Uninstall HORIZON GRID	Standard Inno Setup uninstaller entry point — see "Uninstalling" below.

A desktop icon replicating "Open Platform" is also created if you selected that optional task during installation.

[MISSING FIGURE: standalone-start-menu.png]

Firewall Rule

During "Start Installation," the wizard creates a single inbound Windows Firewall rule (named "HORIZON GRID") allowing TCP traffic on the frontend and backend ports, scoped specifically to the **Private** network profile — never Domain, never Public. This is what makes the platform reachable from other devices on the same trusted home or office network without any manual firewall configuration, while deliberately not exposing it to a public or untrusted network. Reconfiguring ports later removes and recreates this rule with the new port numbers rather than leaving a stale duplicate behind. Uninstalling — either "Remove Application" or "Remove Everything" — always removes this rule, regardless of which uninstall path you choose, since a firewall rule isn't user data.

Data Location

Installed program files live under `%ProgramFiles%\IOC Intelligence Platform\app\` — the backend and frontend source, the Docker Compose files (including `docker-compose.prod.yml`, the production overlay that drops development bind-mounts and switches to

production start commands), and the wizard/operational scripts. This tree is read-mostly; it's what `docker compose` builds images from.

Writable, per-machine data lives separately, under `%ProgramData%\IOC Intelligence Platform\`:

Path	Contents
`config\.env`	The real environment file — database/JWT secrets, AI credentials, provider API keys, port mappings.
`logs\setup.log`	Human-readable wizard/setup log.
`backups\`	Timestamped `pg_dump` snapshots; the 10 most recent are kept, older ones pruned automatically.

Both of these locations keep the internal folder name `IOC Intelligence Platform`, deliberately, even though the product is branded HORIZON GRID everywhere you actually see it — the installer title, the Start Menu group, the wizard window, the web interface. This is not an incomplete rebrand; it's a deliberate choice made specifically to protect upgrade safety. The one real, already-existing install on a given machine has its data sitting under that exact folder name today. If a future version's rebrand touched this internal path, every script this installer relies on (which resolve `$script:InstallDir` and `$script:DataDir` from this hardcoded name, independent of any product-name string) would start looking in the wrong place, and an in-place upgrade would either silently fail to find the existing configuration or, worse, orphan it. The uninstaller's own destructive-removal code path deliberately hardcodes this same folder name rather than deriving it from any rebrand-sensitive constant, for exactly the same reason.

The entire `ProgramData\IOC Intelligence Platform\` tree is locked down to the Administrators and SYSTEM accounts only, via `icacls`, immediately after it's created — no other account on the machine can read the `.env` file's secrets. Because Docker Compose's own `env_file: .env` directive resolves relative to the Compose project directory rather than wherever a `--env-file` flag points, a synchronized working copy of `.env` also exists at `{app}\app\.env`, re-locked to the same Administrators+SYSTEM ACL on every single `docker compose` invocation — by design, there are two on-disk copies of the secrets, and both are protected identically, not just the first one.

Upgrade / Reconfigure

Re-running the **Configuration** shortcut on a machine that already has the platform installed changes the wizard's behavior in two concrete ways, both driven by the wizard detecting an existing `.env` file at startup:

1. **Every page is pre-filled** from the real current configuration — AI backend and model, every provider key, every port, the AI backend's own credentials — so reconfiguring is a review-and-adjust flow, not a blank slate. The Administrator Account page explains that leaving its fields empty keeps the existing account unchanged.
2. **The database is backed up automatically before anything else changes**, by running the same `pg_dump`-in-container mechanism as the "Backup Database Now" shortcut. This happens as the very first step of clicking "Start Installation," before the new configuration is even written — and its real outcome (success or failure) is reported plainly in the install log rather than assumed. A prior version of this step used to print "Backup complete" even when there was nothing to back up yet (for example, retrying after an earlier attempt failed before any container had started); it now reflects the backup script's real exit code instead.

From there, the wizard proceeds through the identical page sequence, and "Start Installation" re-runs the same `docker compose up -d --build` and health-poll sequence used on a fresh install. Your investigation data, cases, watchlists, and history are never touched by anything in this flow — only the settings you actually change in the wizard.

Uninstalling

Launching the uninstaller (Start Menu shortcut, or Windows Settings → Apps) presents a choice before anything is removed:

- **Remove Application (the default, "Yes" option)** — stops the running containers (without the `-v` flag, so volumes and therefore all data survive) and removes the installed program files. Your configuration, credentials, and all investigation data (cases, watchlists, notes, the database itself) are left exactly as they were. Reinstalling later picks up right where you left off.
- **Remove Everything (the "No" option, followed by a second confirmation)** — this is the explicit, destructive opt-in. It requires typing `DELETE` (all capital letters) into a confirmation box before proceeding; a plain Yes/No click is not enough for an action this irreversible. Once confirmed, it runs `docker compose down -v` (the `-v` is what actually deletes the named data volumes — Postgres, Neo4j, OpenSearch) and deletes the entire `ProgramData\IOC Intelligence Platform\` tree. There is no automatic backup taken as part of this path; if you might want the data back, use "Backup Database Now" first, or choose "Remove Application" instead.

A silent/unattended uninstall (`/VERYSILENT`) always takes the non-destructive "keep data" path automatically, since there's no one present to answer the confirmation prompt — "Remove Everything" is reachable only through the interactive uninstall flow, on purpose.

[MISSING FIGURE: standalone-uninstall-confirm.png]

A Real Bug That Was Found and Fixed: The Fresh-Install Password Mismatch

During this project, a real installation failure was reported and root-caused: on a machine where an earlier, abandoned install attempt had left a Postgres data volume behind (for example, an install that was started and then cancelled, or that failed before ever being cleanly uninstalled), a later fresh install would crash-loop the backend container immediately, failing every time on a Postgres authentication error.

The underlying cause was a genuine gap in how the pieces fit together, not a typo or a one-off mistake: every fresh install (one where the wizard finds no existing `.env` to load real values from) generates a brand-new, cryptographically random database password. That's correct and desirable behavior on a truly clean machine. The problem is that Postgres only ever applies the `POSTGRES_PASSWORD` environment variable the *first* time it initializes an empty data directory — once a volume has been initialized once, every later container start ignores that variable entirely and keeps whatever password is already baked into the volume. If a volume from an earlier, abandoned attempt was still sitting on the machine, the new install's freshly generated password would never match it, and the backend would fail to authenticate against its own database from the very first startup.

The fix, in `Common.ps1` 's `Remove-StaleDatabaseVolumeIfFreshInstall` , detects this specific situation and clears it automatically: on a genuinely fresh install only (never on an upgrade or reconfigure of a real existing install, which correctly reuses its real existing password and must never have its volume touched), the wizard looks up any Postgres data volume left behind by Docker Compose's own project/volume labels — not a hardcoded volume name, since that label lookup stays correct even if the underlying project structure ever changes — and removes it before writing the new configuration. Without a matching `.env` , that orphaned volume's data was already unreachable anyway (nothing on the machine still knew its real password either), so removing it trades an inaccessible, silently-broken leftover for a clean, working fresh install; it is not a data-loss risk in any install that was actually completed successfully. The installer has been rebuilt from this fixed source.

See Also

- `user-03-installation.md` — the administrator-facing walkthrough of the first-stage Inno Setup screens and LAN access, in plain-language terms.
- `tech-06-windows-deployment.md` — the architectural view: the Program Files/ProgramData split, ACL protections, and what "Start Installation" executes under the hood.